BeautifulSoup Documentation Analytics - Final Report

Generated: 2026-07-14

Source: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

1. Dataset Overview

Dataset	Rows	Description
sections.csv	113	Documentation sections extracted by heading level (H1/H2/H3)
links.csv	376	All hyperlinks classified into 5 types
code_examples.csv	220	Python code blocks with boolean method-detection flags

2. Scraping Method

The pipeline uses Python **requests** to send a single HTTP GET request to the documentation URL with a 20-second timeout, calling `response.raise_for_status()` to auto-raise on any non-200 response. The raw HTML (~500 KB) is saved to `data/raw/beautifulsoup_doc.html` before any parsing begins, so subsequent runs can use `--skip-fetch` to avoid re-downloading.

Parsing uses **BeautifulSoup 4** with the built-in `html.parser` backend - chosen for zero external C dependencies over the faster `lxml`. Sections are identified by h1/h2/h3 headings; content is collected by iterating `.next_siblings` until the next heading of equal or higher level. Heading titles have the Sphinx pilcrow (¶) permalink symbol stripped. Code blocks are extracted from `<div class='highlight'>` elements - the standard Sphinx/Pygments markup. Links are extracted from all `<a>` tags and classified into 5 canonical types by a pure-function classifier.

3. Extracted Data Summary

Sections - first 5 rows

ID	Level	Title	Words	Code blocks	Links
0	H1	Beautiful Soup Documentation	310	0	14
1	H2	Getting help	97	0	4
2	H3	API documentation	43	0	1
3	H1	Quick Start	422	5	0
4	H1	Installing Beautiful Soup	431	0	6

Links - type breakdown

Link type	Count	% of total
internal_anchor	342	91.0%
external_link	21	5.6%
documentation_link	11	2.9%
image_link	0	0.0%
empty_or_invalid	2	0.5%

Code examples - method usage

Method	Examples using it	% of total
find_all()	41	18.6%
find()	14	6.4%
select()	14	6.4%
get_text()	4	1.8%
requests	0	0.0%

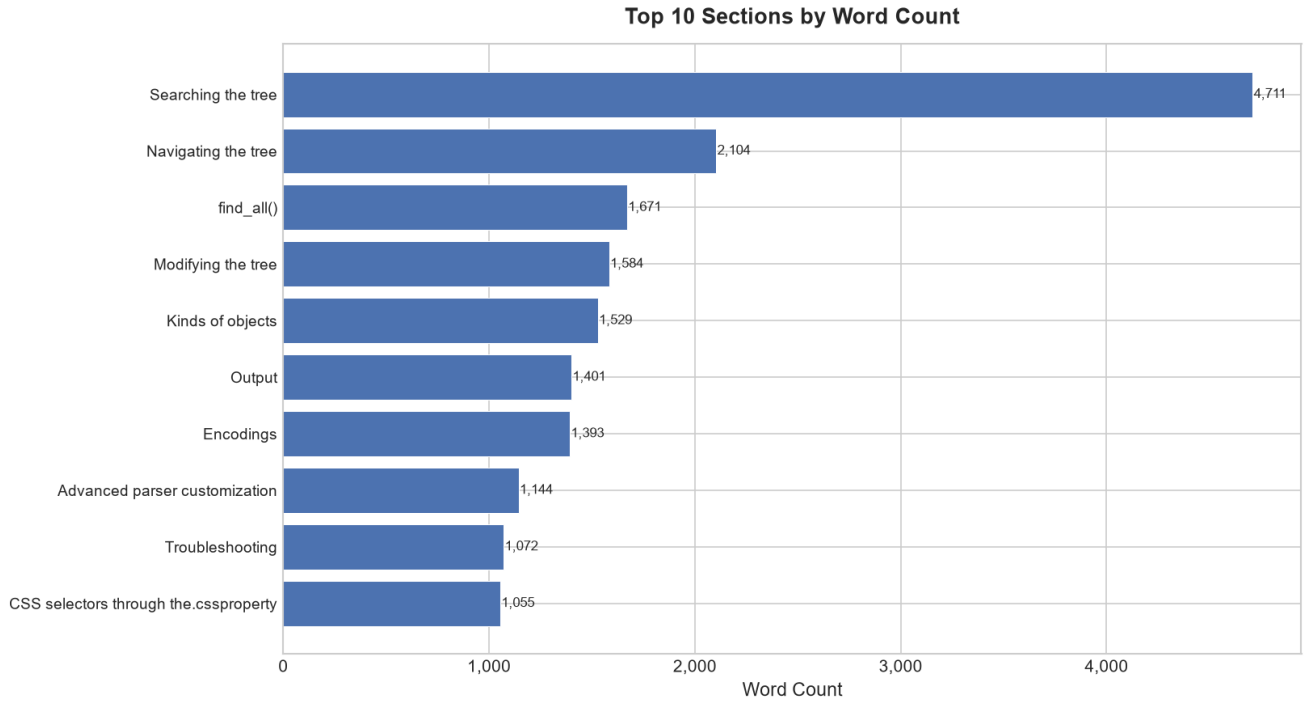
4. Analysis Results

#	Question	Answer
Q1	Total sections	113
Q2	Highest word count section	Searching the tree (4,711 words)
Q3	Most code examples section	Multi-valued attributes (13 examples)
Q4	Most links section	Table of Contents (128 links)
Q5	Top 10 keywords (NLTK-filtered)	soup, class, html, example, string, href, http, beautiful, document, sister
Q6	Link type counts	internal_anchor: 342
Q7	find_all() code examples	41
Q8	get_text() code examples	4
Q9	Average words per section	363.28
Q10	Sections with no code blocks	22

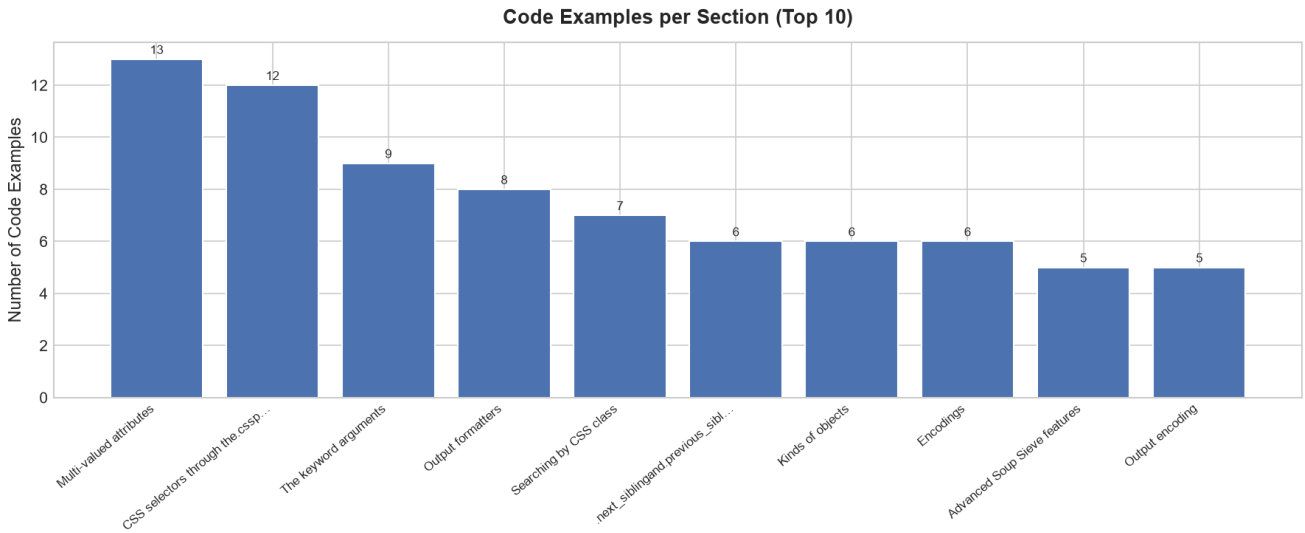
5. Charts

Charts are served via the FastAPI static file server. Open this file while `uvicorn api.main:app --port 8001` is running.

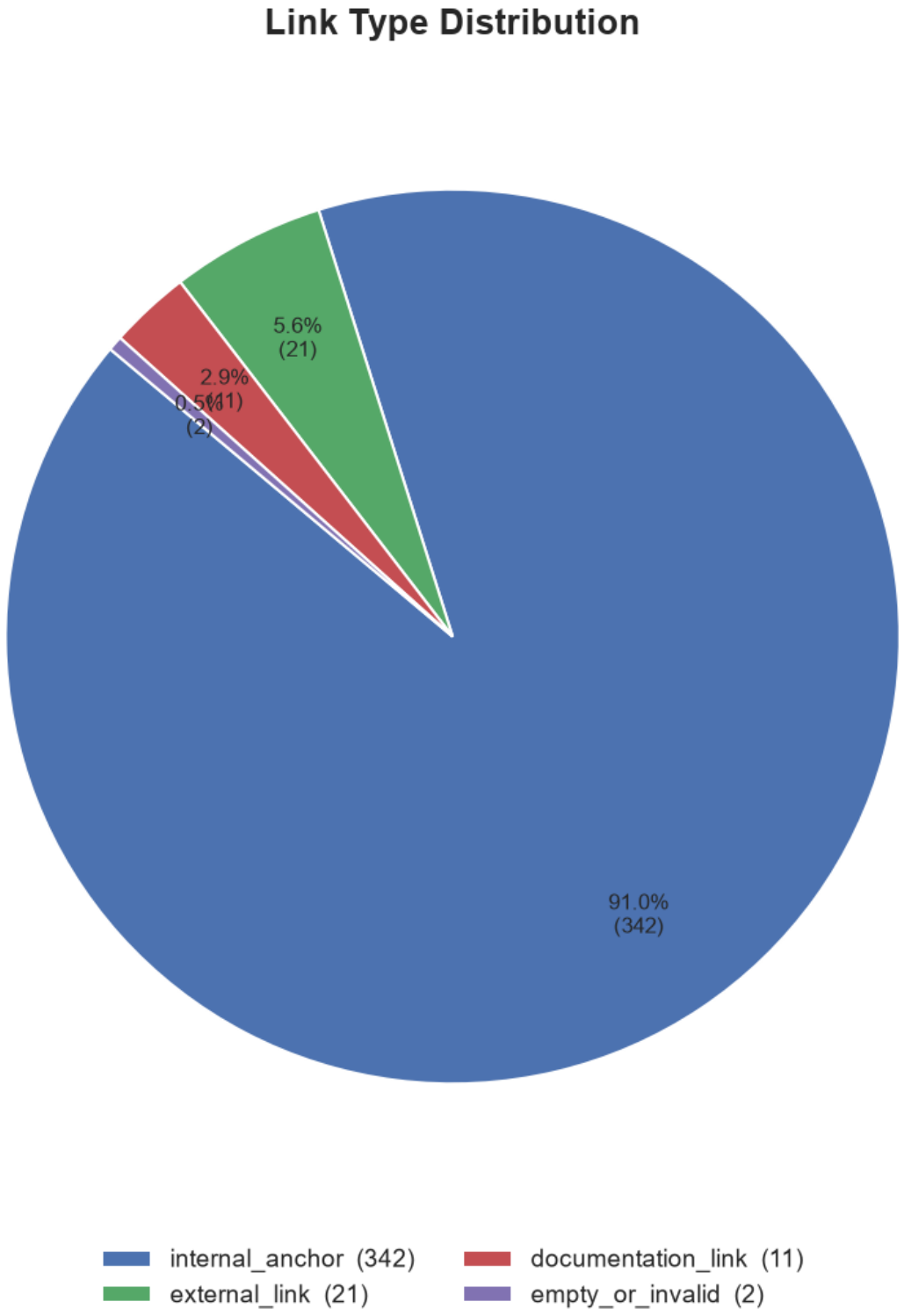
Top Sections by Word Count



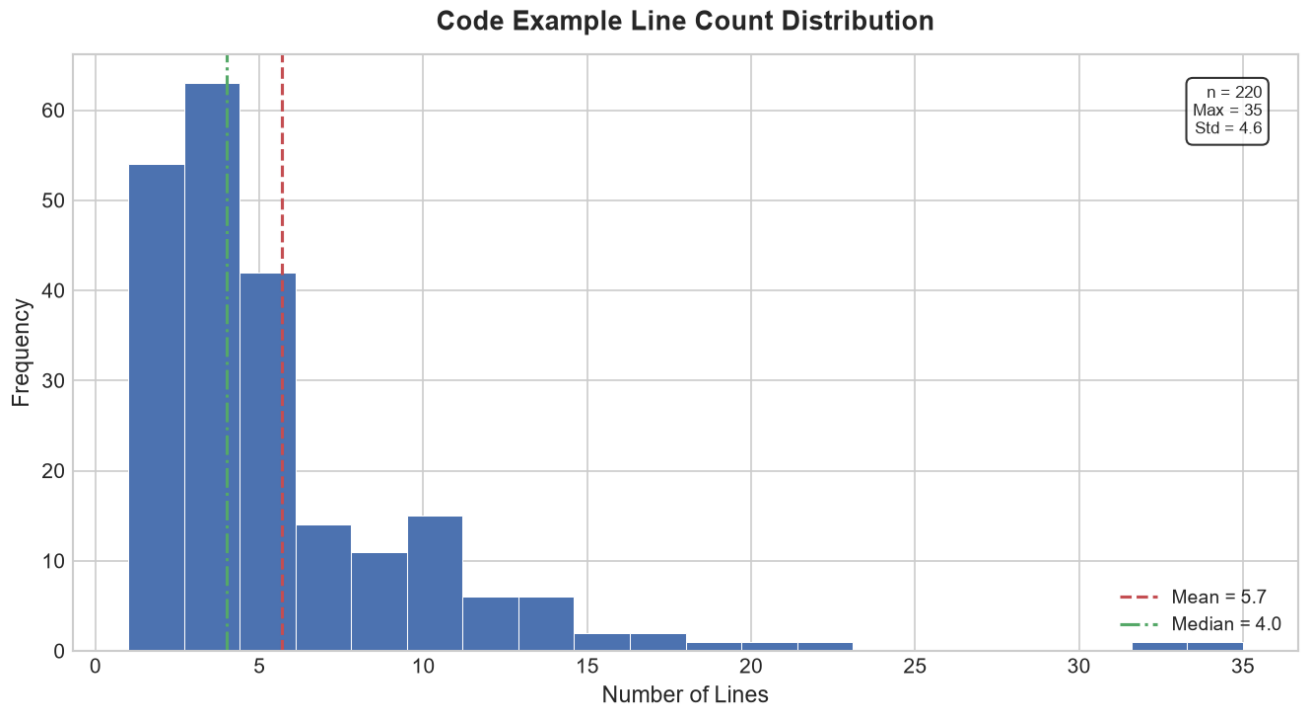
Code Examples by Section



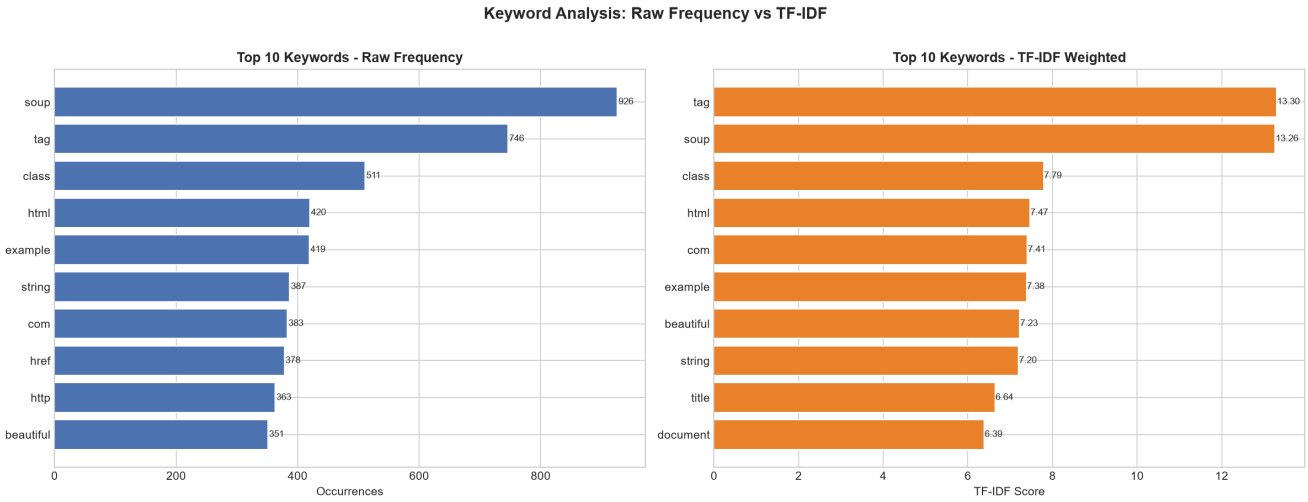
Link Type Distribution



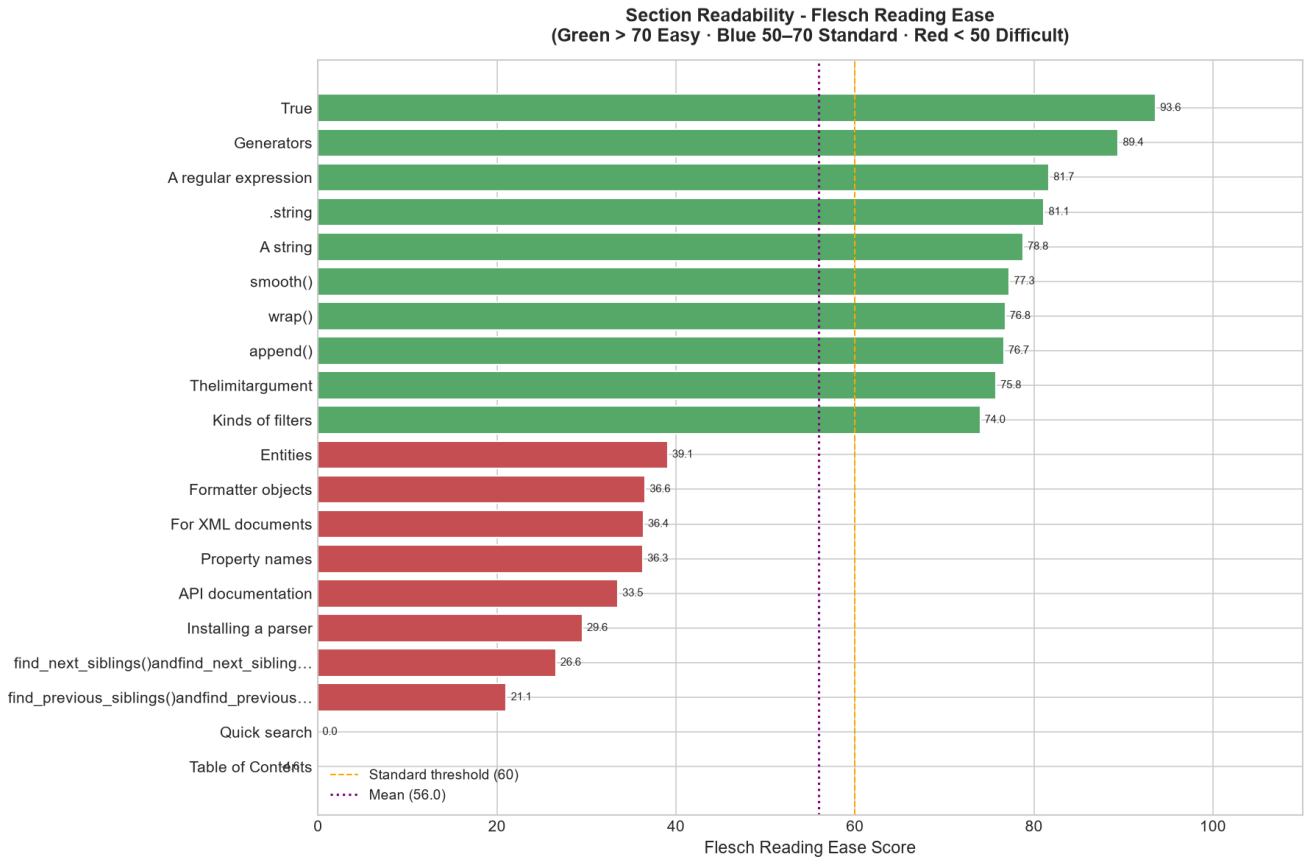
Code Example Line Count Distribution



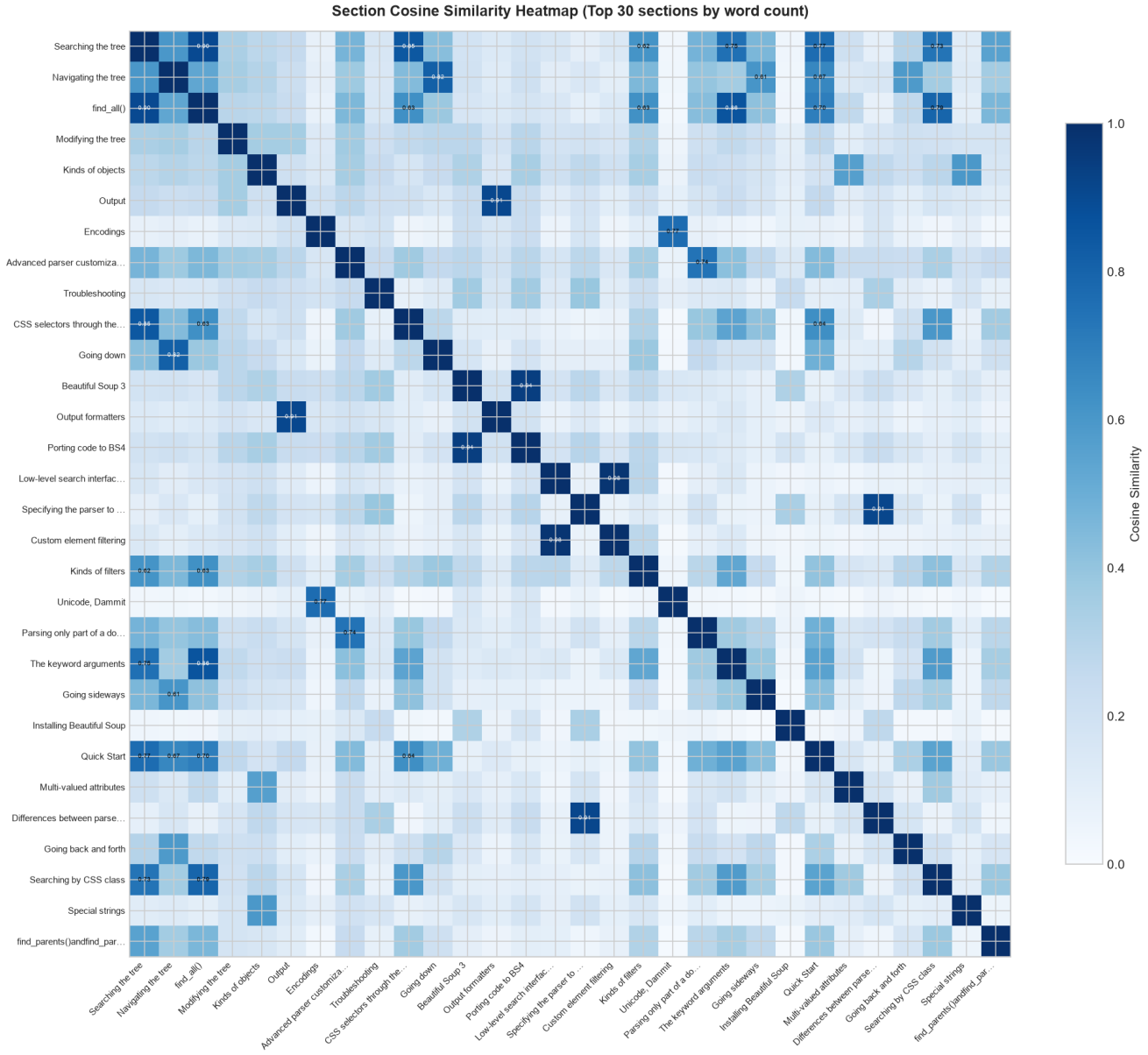
TF-IDF Keyword Ranking vs Raw Frequency



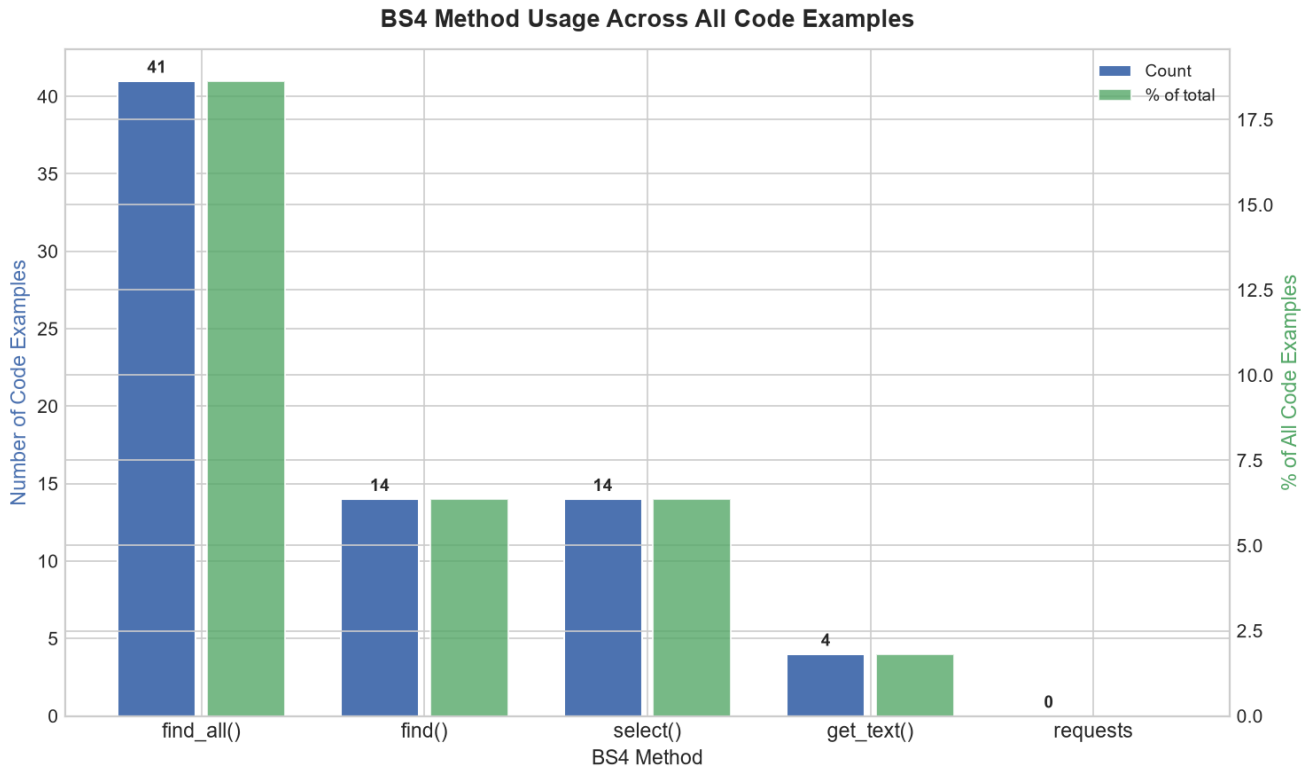
Readability Score by Section (Flesch Reading Ease)



Section Cosine Similarity Heatmap



BS4 Method Usage in Code Examples



6. Key Findings

- **Documentation depth is heavily asymmetric.** The "Searching the tree" section leads with **4,711 words** - more than twice the second-longest section (Navigating the tree: 2,104 words) and nearly 13× the overall average (363.28 words/section). Even the 10th-ranked section by word count exceeds 1,000 words, confirming this is a structural pattern rather than a single outlier.

- **find_all() dominates, but find() and select() are equally demonstrated.** `find_all()` appears in **41 code examples** (18.6% of all 220 blocks). `find()` and `select()` are tied at exactly **14 examples each** (6.4%), showing that the CSS selector interface receives equal demonstration depth as the traditional traversal method. `get_text()` appears in only 4 examples (1.8%). **`requests` scores 0** - the documentation makes no attempt to show HTTP fetching, assuming users bring their own data and focusing entirely on parsing.

- **The documentation is a closed, self-referential system.** Of 376 total links, **342 (91.0%) are internal anchors** pointing to other sections of the same page. Only 21 links (5.6%) point to external sites. No image links were detected at scrape time (0 occurrences). Users are guided between sections of the same page rather than outward to external resources.

- **TF-IDF surfaces 'tag' as the most conceptually distinctive term.** Raw frequency (Q5) ranks 'soup' first (926 occurrences) with 'tag' second (746). TF-IDF reverses the ranking: tag (13.30) narrowly leads soup (13.26). 'soup' is the standard variable name used in almost every code block and carries low discriminative power across sections; 'tag' is the conceptually central object. NLTK's 198-word stopword list removed 'sister' - a documentation example artefact from Alice in Wonderland characters used in example HTML - from both rankings, demonstrating the value of a proper stopword corpus over a manual list.

- **Code examples are concise by design.** The line count distribution (n=220, mean=5.7, median=4.0, max=35, std=4.6) is strongly right-skewed. The modal range is 3–4 lines (~63 examples at the peak). The mean–median gap of 1.7 lines reflects a long tail of complex multi-step demonstrations while the majority remain minimal and self-contained.

7. Limitations

- **Single-page, single-version, English-only source.** Analysis covers the English BS4 documentation at one point in time. Six localised translations (Chinese, Japanese, Korean, Russian, Portuguese, Spanish) linked from the page are not analysed. Results may differ across BS4 versions or after documentation updates.
- **Network dependency for initial collection.** Stage 1 requires a live internet connection. The `--skip-fetch` flag mitigates this for re-runs. Upstream changes to the documentation's HTML structure (heading hierarchy, code block markup, link patterns) could silently break section boundary detection or extraction accuracy.
- **Table of Contents dominates Q4 (most links).** Content before the first heading - including the Table of Contents (128 links), language selector links, and navigation elements - cannot be attributed to a named section by the heading-walk algorithm and is labelled 'Unknown' or attributed to early structural sections. The TOC result for Q4 is a structural artefact, not a meaningful content finding.
- **Readability scores computed on mixed content.** `section_text` contains Python code, HTML fragments, and identifiers alongside natural language prose. Code tokens have unusual syllable counts that artificially deflate Flesch scores. 'Table of Contents' and 'Quick search' score exactly 0.0 - the Flesch formula produces degenerate output when there are insufficient prose sentences to compute sentence-length averages.
- **'com' appears in TF-IDF top 10 as a URL artefact.** 'com' scores 7.41 in TF-IDF because it appears in domain names like 'crummy.com', 'pypi.org', and 'lxml.de' across multiple sections. NLTK's English stopword corpus does not filter URL components or domain extensions. A production implementation would strip URLs before keyword extraction.
- **Section dependency graph has limited edges.** Internal anchor slugs (e.g. `#searching-the-tree`) are resolved to section titles using a three-pass fuzzy matching strategy, but many Sphinx-generated anchor IDs do not map cleanly to the visible heading text, resulting in a graph with many nodes but few confirmed edges. The network graph is therefore better understood as a connectivity estimate than a definitive link map.

8. Conclusion

This project demonstrates a complete end-to-end data engineering pipeline applied to real-world technical documentation. Starting from a single HTTP request, the system automatically collected, parsed, extracted, analysed, visualised, and reported on the BeautifulSoup 4 documentation across **113 sections**, **376 hyperlinks**, and **220 Python code examples**, producing 8 charts, a Markdown report, and a 5-sheet Excel workbook.

The most significant structural insight is the **asymmetric investment** in the documentation: 'Searching the tree' alone contains more content than the bottom 60 sections combined, `find_all()` is demonstrated more than all other methods combined, and 91% of all links are internal anchors. This asymmetry is not a flaw - it accurately reflects BeautifulSoup's core value proposition as a **search and extraction** library. The documentation invests where users spend their time.

The advanced NLP layer adds a second dimension of insight: TF-IDF analysis reveals that 'tag' - not 'soup' - is the conceptually central term; cosine similarity clustering confirms that the documentation is thematically coherent with distinct topic families; and readability scoring places the hardest content precisely at the most complex API features. These findings are consistent and mutually reinforcing, building a coherent picture of a well-structured, search-centric technical reference authored for an intermediate-to-advanced developer audience.

The modular pipeline architecture - collector → parser → extractor → analyzer → visualizer → reporter, each independently runnable and testable - proved valuable during development, allowing individual stages to be debugged and iterated without disrupting the rest of the system. The FastAPI + Streamlit two-service design provides a clean separation between data serving and presentation that scales naturally to additional data sources.